

An aerial photograph of the University of British Columbia Okanagan (UBCO) campus. The image shows several large, modern brick buildings with flat roofs, some featuring solar panels. The campus is surrounded by lush green trees and lawns. In the background, there are rolling hills and mountains under a clear blue sky with a few wispy clouds. The lighting suggests it's either early morning or late afternoon, with long shadows cast across the landscape.

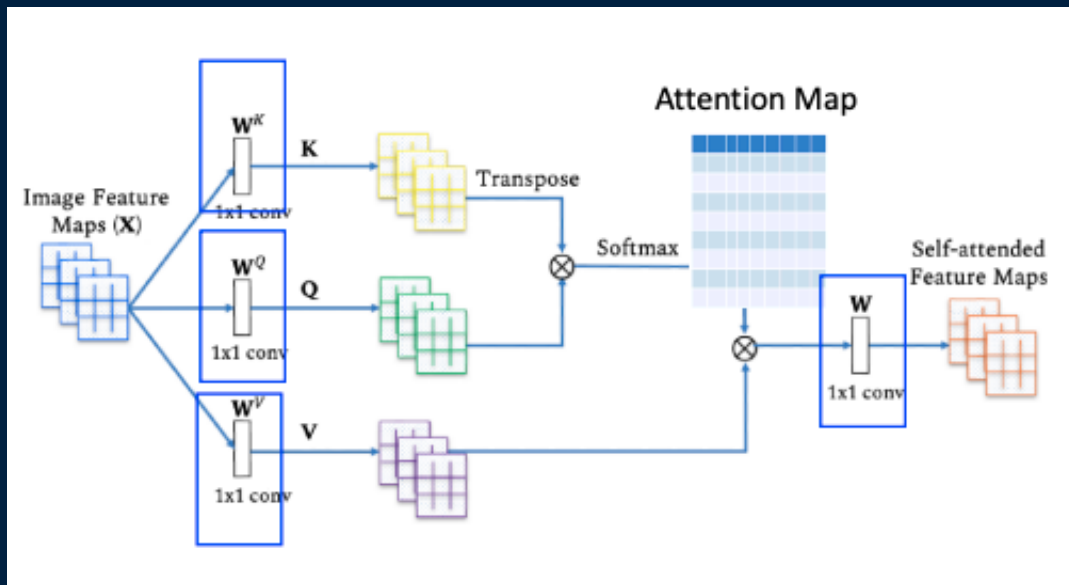
Special Topics: Applied Computer Vision and Deep Learning

UBCO Master of Data Science – DATA 589



Self Attention for Image

- The **Queries** represent *what each location is looking for*.
- The **Keys** represent *what each location offers*.
- The **Values** represent *the information that will be shared*.





Self Attention for Image

n = number of entities

- words in NLP
- pixels / patches in vision

d = feature dimension

In vision:

Each x_i is a **feature vector at one spatial location**

Learnable projection matrices

$$W^Q \in \mathbb{R}^{d \times d_q}, W^K \in \mathbb{R}^{d \times d_k}, W^V \in \mathbb{R}^{d \times d_v}$$

• These are **learnable parameters**

• They define *what kind of similarity* the model cares about.

In Attention mechanism: We have

$$Q = XW^Q, K = XW^K, V = XW^V$$

These are **attention projections/linear projections** that applied to each token (or pixel)

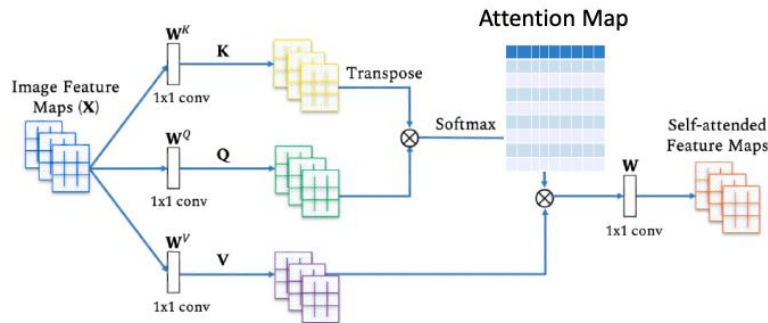
Lets denote a sequence of n entities ($x_1, x_2, \dots, x_n$)

by $\hat{X} \in \mathbb{R}^{n \times d}$,

Queries ($W^Q \in \mathbb{R}^{d \times d_q}$), Keys ($W^K \in \mathbb{R}^{d \times d_k}$) and Values ($W^V \in \mathbb{R}^{d \times d_v}$), where $d_q = d_k$. The input sequence X is

first projected onto these weight matrices to get $\hat{Q} = XW^Q$, $K = XW^K$ and $V = XW^V$. The output $Z \in \mathbb{R}^{n \times d_v}$ of the

$$Z = \text{softmax} \left(\frac{QK^T}{\sqrt{d_q}} \right) V.$$



Self-Attention - image

Attention projections W^Q, W^K, W^V :

- Learn how to represent tokens
- Produce feature embeddings, not feature maps (like CNNs)

Given input: $3 \times 3 \times 3$

We assume the format is: $H \times W \times C = 3 \times 3 \times 3$

Height = 3

Width = 3

Channels = 3

This is a **very small image / feature map**.

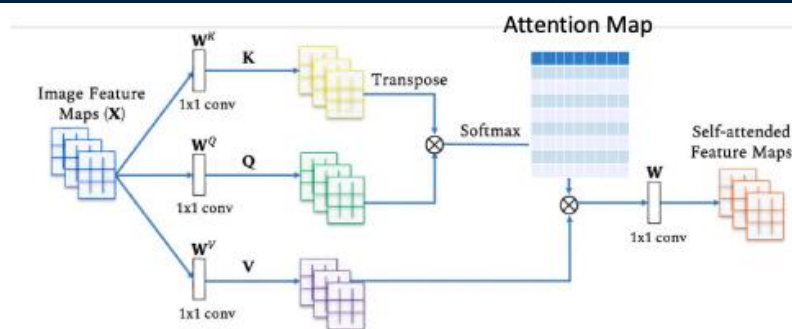
by $\tilde{X} \in \mathbb{R}^{n \times d}$
 9×3

Queries ($W^Q \in \mathbb{R}^{d \times d_q}$), Keys ($W^K \in \mathbb{R}^{d \times d_k}$) and Values ($W^V \in \mathbb{R}^{d \times d_v}$), where $d_q = d_k$. The input sequence X is 3×3

first projected onto these weight matrices to get $Q = XW^Q$, $K = XW^K$ and $V = XW^V$. The output $Z \in \mathbb{R}^{n \times d_v}$ of the 9×3

$$Z = \text{softmax} \left(\frac{QK^T}{\sqrt{d_q}} \right) V.$$

9×3 9×9 9×3



Self-Attention - image

Assume the input to attention is $X \in \mathbb{R}^{n \times d} = 9 \times 3$
 Feature dimension per token: $d = 3$

9 tokens, each with a 3-dimensional feature vector!

Now we apply three different 1×1 convolutions to produce Query, Key, and Value. A 1×1 convolution is just a linear layer that mixes channels, not spatial locations.

Since this is a teaching example, we will **choose**:

$$d_q = d_k = d_v = 3$$

This keeps the math simple:

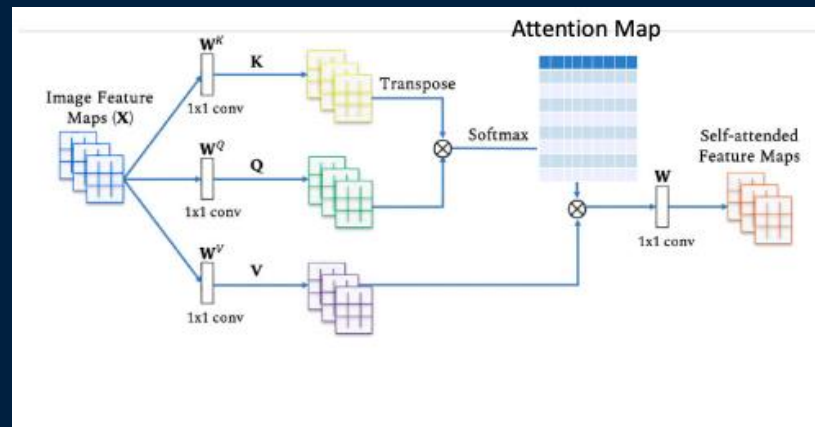
$$W^Q \in \mathbb{R}^{3 \times d_q}, W^K \in \mathbb{R}^{3 \times d_k}, W^V \in \mathbb{R}^{3 \times d_v}$$

Weight matrix dimensions:

$$W^Q \in \mathbb{R}^{3 \times 3}$$

$$W^K \in \mathbb{R}^{3 \times 3}$$

$$W^V \in \mathbb{R}^{3 \times 3}$$



Self-Attention - image

Assume the input to attention is $X \in \mathbb{R}^{n \times d} = 9 \times 3$
 Feature dimension per token: $d = 3$

Compute attention projections or Q, K, V which project the input:

$$\begin{aligned} Q &= XW^Q \\ K &= XW^K \\ V &= XW^V \end{aligned}$$

Dimensions

$$X \in \mathbb{R}^{9 \times 3} \times W^{(\cdot)} \in \mathbb{R}^{3 \times 3}$$

So:

$$Q, K, V \in \mathbb{R}^{9 \times 3}$$

✓ Each of the 9 tokens now has:

- a Query vector
- a Key vector
- a Value vector

Weight matrix dimensions:

$$W^Q \in \mathbb{R}^{3 \times 3}$$

$$W^K \in \mathbb{R}^{3 \times 3}$$

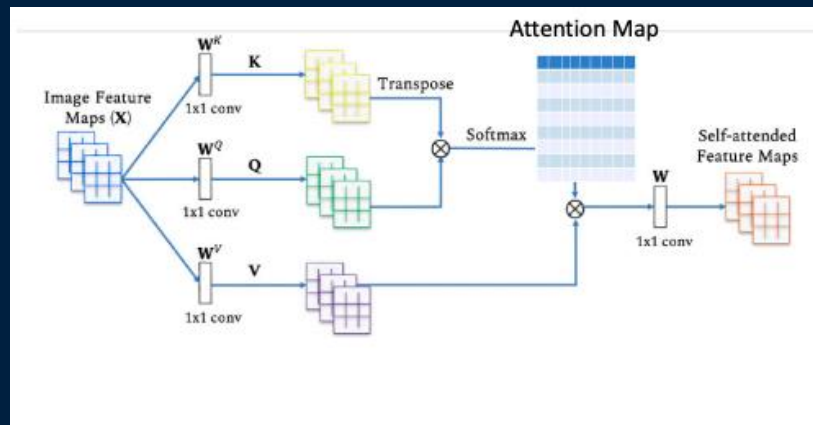
$$W^V \in \mathbb{R}^{3 \times 3}$$

Each one:

Takes a **3D vector**

Outputs a **new 3D vector**

👉 These are **linear transformations**



Self-Attention - image

$$Q, K, V \in \mathbb{R}^{9 \times 3}$$

To compute similarities, we transpose K:

$$K^T \in \mathbb{R}^{3 \times 9}$$

Compute attention scores

$$QK^T$$

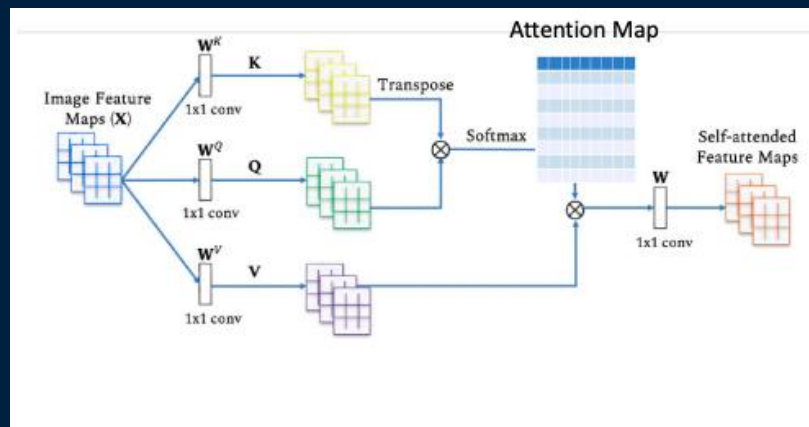
Dimensions

$$(9 \times 3) \times (3 \times 9) = 9 \times 9$$

So:

$$QK^T \in \mathbb{R}^{9 \times 9}$$

Each row corresponds to one token asking how much it should attend to all other tokens.



Self-Attention - image

$$Q, K, V \in \mathbb{R}^{9 \times 3}$$

$$QK^T \in \mathbb{R}^{9 \times 9}$$

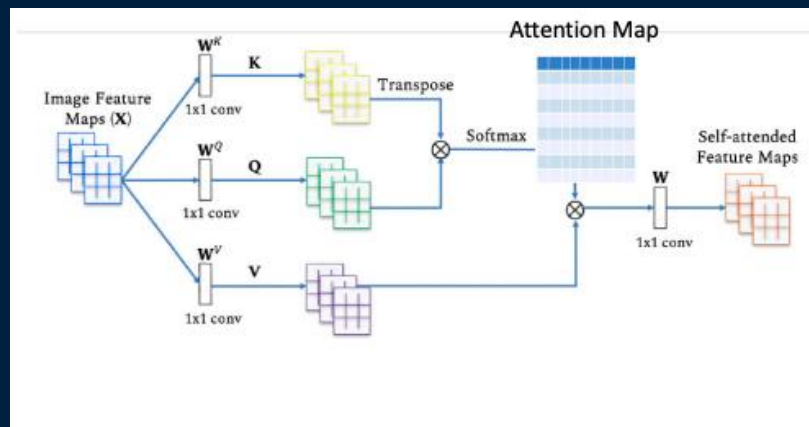
Scale and apply softmax (Attention Map)

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{3}}\right)$$

Dimensions

$$A \in \mathbb{R}^{9 \times 9}$$

- Rows sum to 1
- Shows how attention is distributed



Self-Attention - image

$$Q, K, V \in \mathbb{R}^{9 \times 3}$$

$$QK^T \in \mathbb{R}^{9 \times 9}$$

$$A \in \mathbb{R}^{9 \times 9}$$

Apply attention to Values

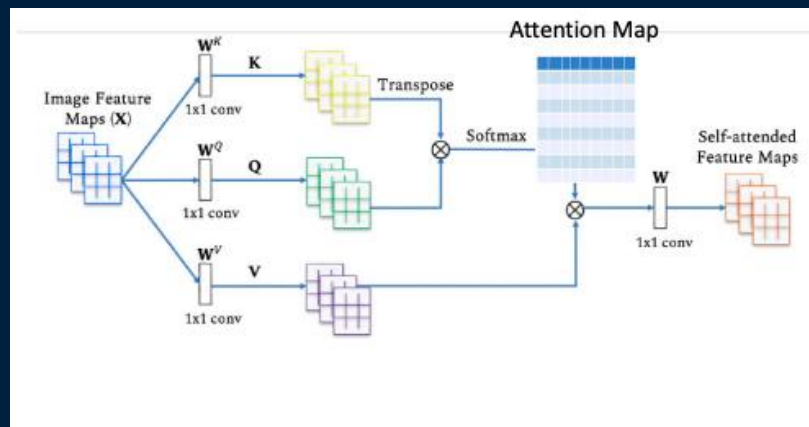
$$Z = AV$$

Dimensions

$$(9 \times 9) \times (9 \times 3) = 9 \times 3$$

So:

$$Z \in \mathbb{R}^{9 \times 3}$$



Self-Attention - image

$$Q, K, V \in \mathbb{R}^{9 \times 3}$$

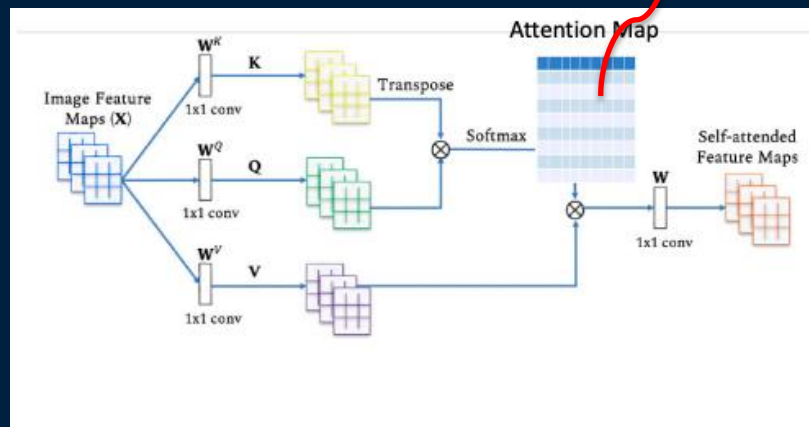
$$QK^T \in \mathbb{R}^{9 \times 9}$$

$$A \in \mathbb{R}^{9 \times 9}$$

$$Z \in \mathbb{R}^{9 \times 3}$$

Each row tells us how one pixel distributes its attention budget across the entire image.

Instead of only looking at neighbors like a convolution, every pixel can look everywhere — but it learns *how much* to care.



Self-Attention - image

$$Q, K, V \in \mathbb{R}^{9 \times 3}$$

$$QK^T \in \mathbb{R}^{9 \times 9}$$

$$A \in \mathbb{R}^{9 \times 9}$$

$$Z \in \mathbb{R}^{9 \times 3}$$

Optional output projection (final 1x1 convolution)

Often we apply another learnable projection:

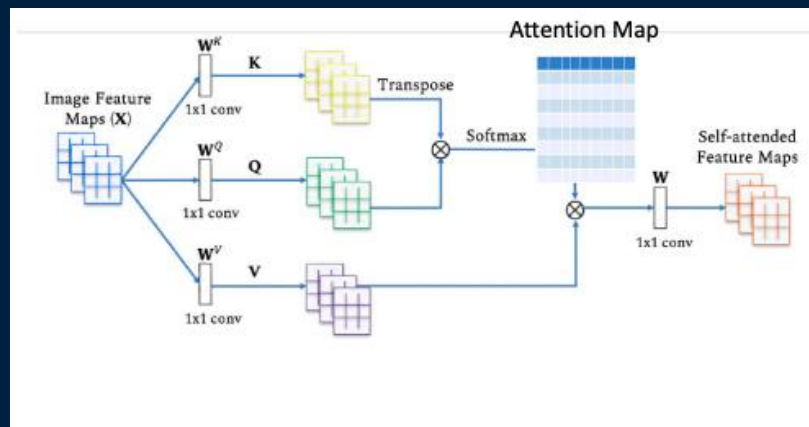
$$W^O \in \mathbb{R}^{3 \times 3}$$

$$Y = ZW^O$$

Dimensions: $(9 \times 3) \times (3 \times 3) = 9 \times 3$

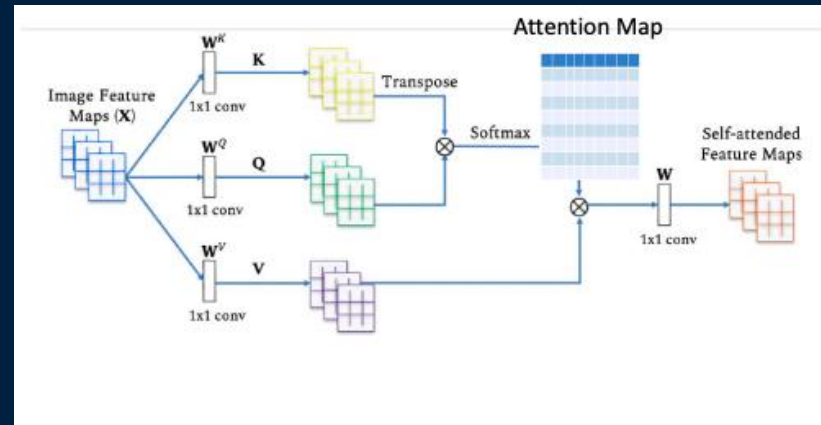
So: $Y \in \mathbb{R}^{9 \times 3}$

This allows: Channel mixing and Residual connection with X



Self-Attention - image

Quantity	Dimension
Input image	$(3 \times 3 \times 3)$
Tokens (n)	9
Input X	9×3
W^Q, W^K, W^V	3×3
Q, K, V	9×3
QK^T	9×9
Attention map	9×9
Z	9×3
Output Y	$3 \times 3 \times 3$



Question

Question: What does a 1×1 convolution do in self-attention?

- A. Extracts spatial patterns
- B. Reduces image size
- C. Detects edges
- D. Applies pooling
- E. Mixes information across channels only

Multi-Head Self-Attention

1. Same input tokens X go into every head.

- Each head sees the **same** X , but learns different projections.

2. Head 1 does self-attention independently

- It computes its own Q_1, K_1, V_1 (via its own learnable linear layers)
- Produces output Z_1

3. Head 2 does the same in parallel

- Its own Q_2, K_2, V_2
- Produces output Z_2

4. Concat outputs across heads

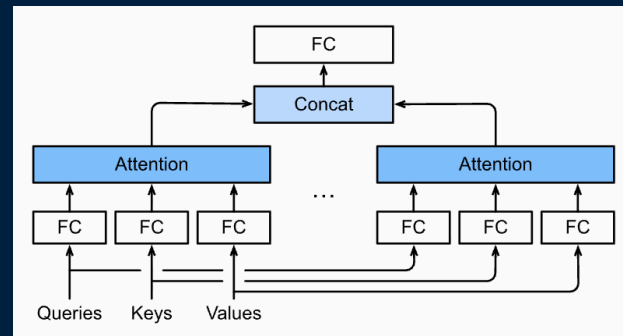
- We stack them side-by-side:

$$Z = [Z_1 \parallel Z_2]$$

- Shape becomes $n \times (h \cdot d_{head})$

5) Output projection W^O

- A final linear layer mixes information across heads and produces the final representation Y
- This makes dimensions compatible for the next block (and residual connection)

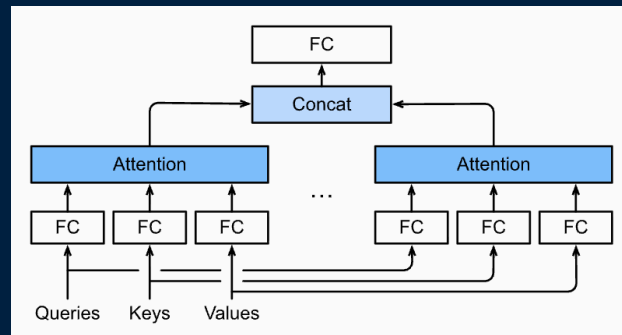


Multi-head attention lets the model attend to different relationships in parallel, using multiple learned subspaces.

Multi-Head Self-Attention (MSA)

For an image:

- Head 1 might focus on local neighborhood structure
- Head 2 might focus on long-range dependencies
- Head 3 might focus on texture or color similarity
- Head 4 might focus on object-level relations



Note: The model does not know this in advance — it *learns* it.

What a single head cannot do well

A single attention head:

- Produces one attention map
- Encodes one dominant type of relationship

Each head attends to the same tokens, but in a different learned subspace!

Compare MSA and CNN

CNN filters

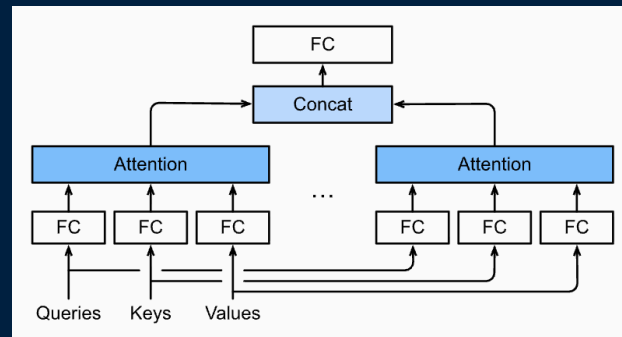
Like colored lenses:

- Each filter highlights a different visual feature
- The neighborhood is fixed

Attention heads

Like different conversation rules:

- One head listens to nearby pixels
- Another head listens to similar textures far away
- Another head listens to object-level structure
- Same input — different interaction patterns.

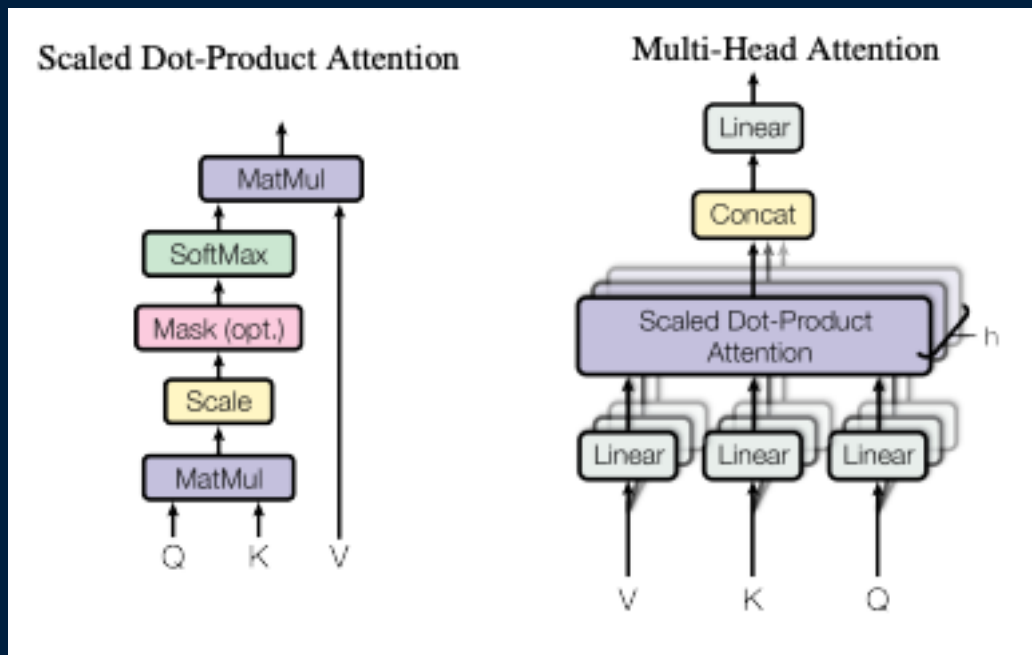


Increasing CNN channels adds more local feature detectors, whereas increasing attention heads adds more ways to model global relationships between tokens.

Recap

Up to this point, we've only talked about **attention as an operation**.

We haven't yet discussed how it fits into a full network — and that's exactly what we'll do next with the Transformer encoder.



Attention is all you need by A Vaswani NIPS 2017

Question

Question: **What is the benefit of multi-head attention?**

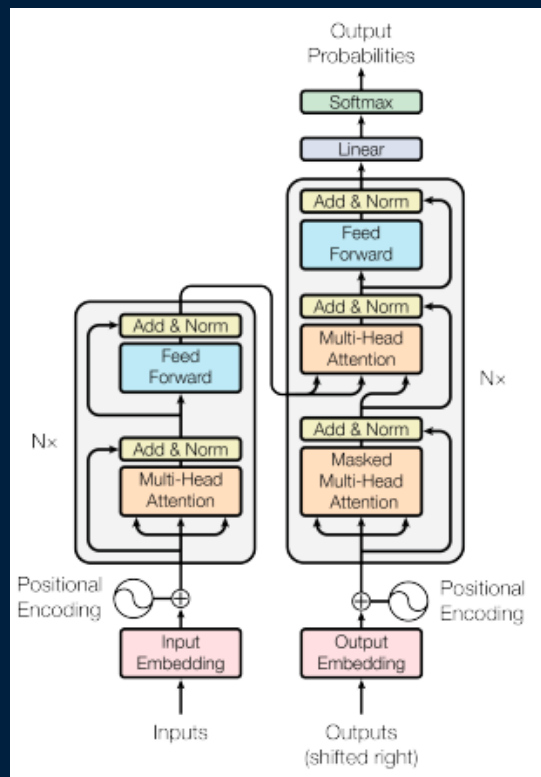
- A. Reduces model size
- B. Processes images faster
- C. Allows the model to capture multiple types of relationships
- D. Removes need for CNN
- E. Reduces number of tokens

Transformer Encoder

The Transformer - model architecture.

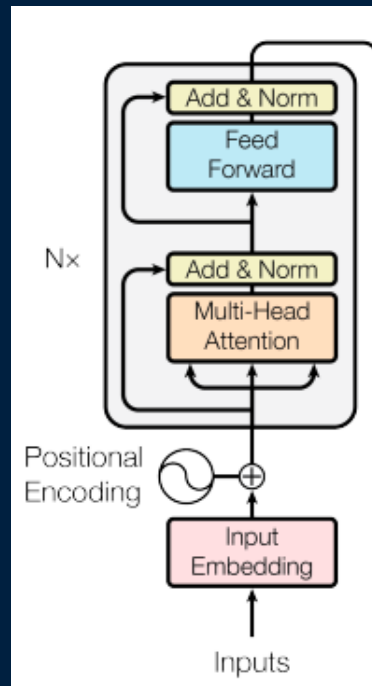
This diagram shows the complete Transformer architecture. For Vision Transformers, we will focus almost entirely on the **left side: the encoder**.

Transformers: Self-Attention + MLP,
not Attention alone



Transformer Encoder

- **Multi-Head Self-Attention:** Nothing new is happening here
 - this is the same attention we already understand.
- **Add & Norm:** After attention, we add the original input back to the output. This is called a **residual connection** to:
 - Stabilizes training
 - Preserves information
 - Helps gradients flow
- **Feed-Forward Network (MLP):** this block is a simple MLP applied **independently to each token**.



Transformer Encoder

Feed-Forward Network (MLP):

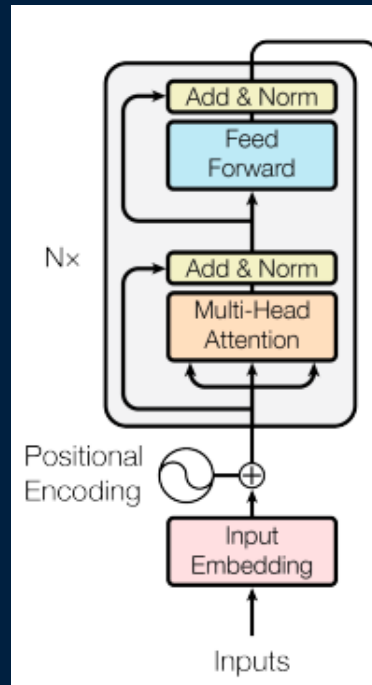
Self-attention is **linear** with respect to values.

Without the MLP:

- The model would be severely limited in expressiveness.
- Stacking attention layers alone cannot model complex transformations.

Attention mixes information *between* tokens.

- The feed-forward network processes information *within* each token.
- In CNNs, we alternate between convolution (spatial mixing) and nonlinearity. In Transformers, we alternate between attention (token mixing) and MLP (feature mixing).



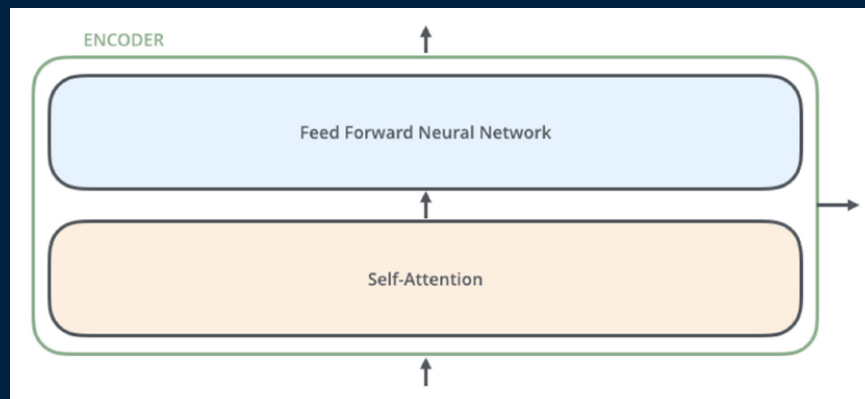
Attention is all you need by A Vaswani + NIPS 2017

Transformer Encoder

A transformer encoder has two main components:

1. Self-Attention
2. Feed Forward Neural Network

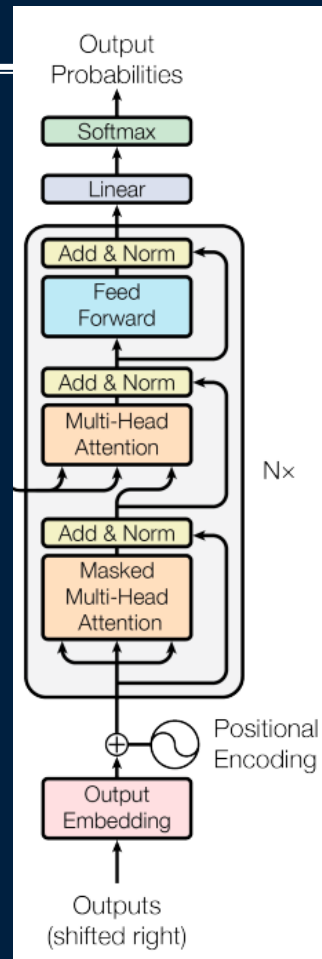
This entire encoder block is stacked multiple times — typically 6, 12, or more layers.



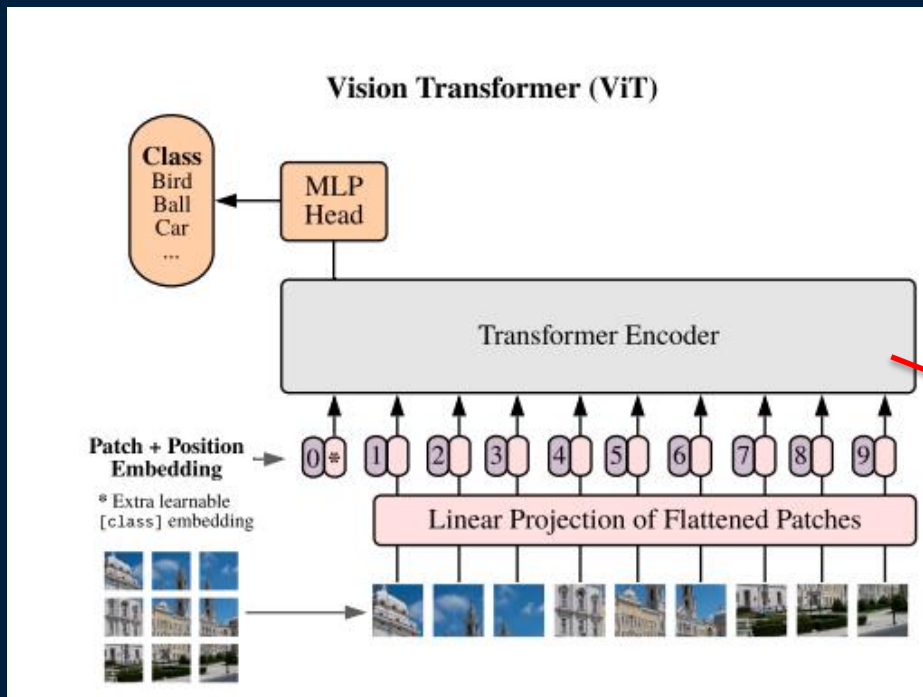
Transformer Decoder

Decoder: used mainly for sequence generation, like machine translation.

For **Vision Transformers**, we **do not use the decoder**, so we won't focus on this part.



Vision Transformer - ViT



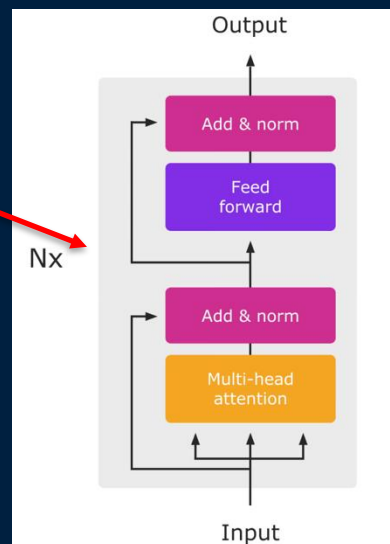
AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com



“AN IMAGE IS WORTH 16X16 WORDS” by Alexey Dosovitskiy +, ICLR 2021

Key Idea

- Applying self-attention directly to pixels is **computationally expensive**.
- ViT solves this by operating on **image patches as tokens**.

Performance (Pretrained on JFT-300M)

- 88.55% on ImageNet
- 90.72% on ImageNet-Real
- 94.55% on CIFAR-100
- 77.63% on VTAB (19 tasks)

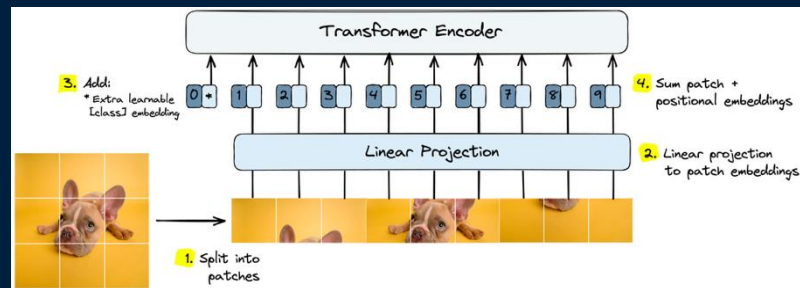
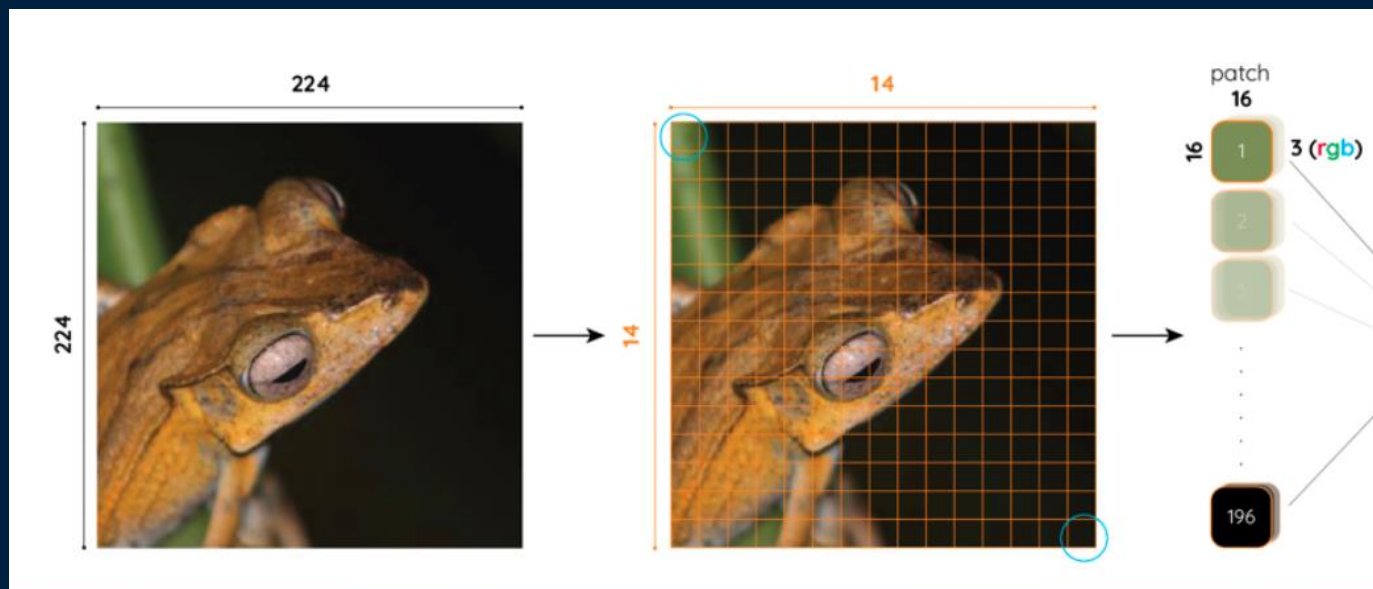


Image of size (3,224,224)

Divided into 196(14×14)

patches of size 3×16×16

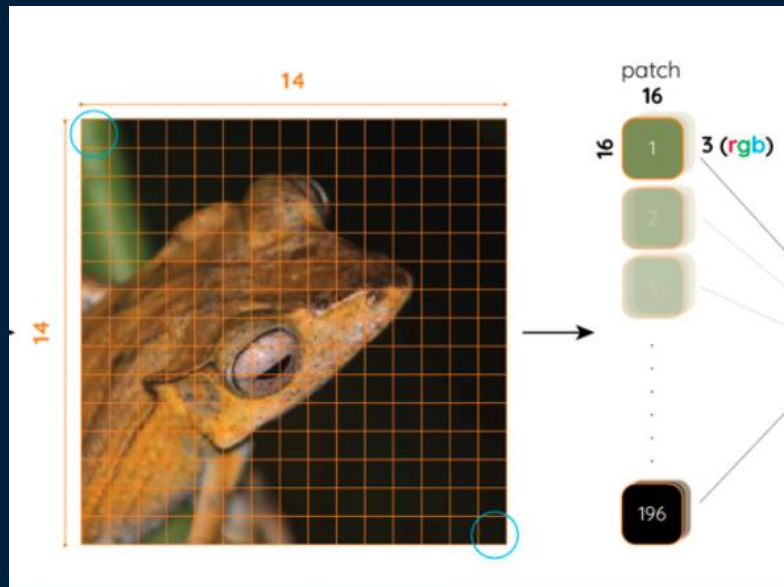
$D16 \times 14 = 224$ (original image size)



Patch Embeddings:

Each patch is a vector of : $3 \times 16 \times 16 = 768$

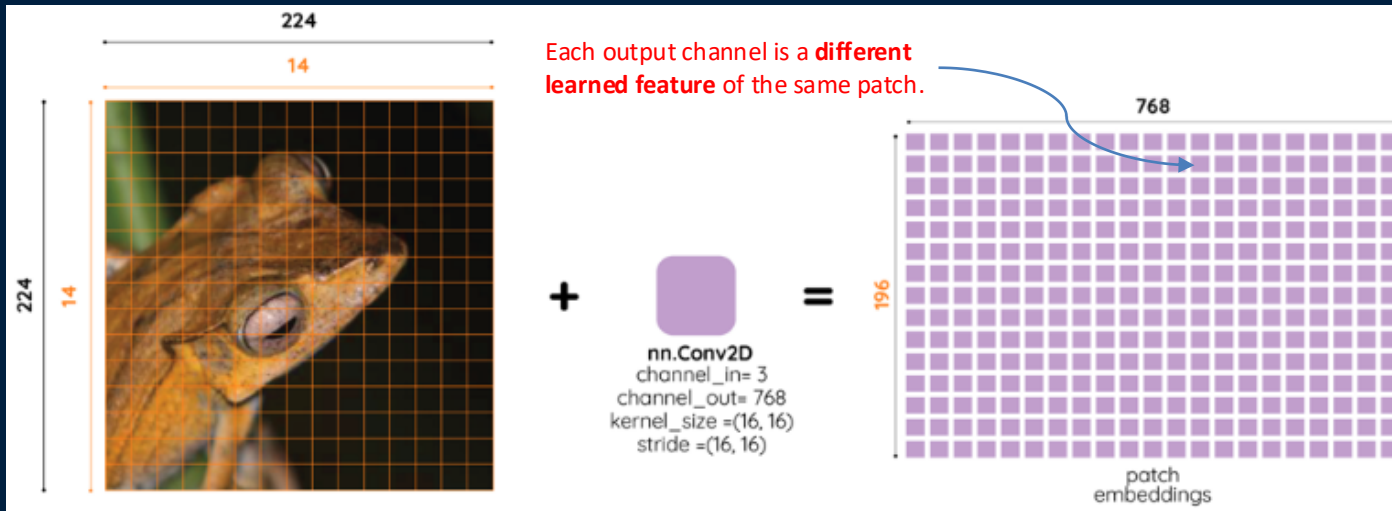
That means this patch embeddings will generate 196 vectors of size 768 or a matrix of size (196, 768)



Generate embedding for each patch

Input image = [3, 224, 224]

Output spatial size = [768, 14, 14]



Note: this is NOT simple flattening; it is a learnable projection (feature extraction) implemented with convolution filters

```
x = torch.randn(1, 3, 224, 224) # 2D conv
nn.Conv2d(in_channels=3, out_channels=768, kernel_size=16, stride=16)
conv(x).reshape(-1, 196).transpose(0,1).shape
```

Question

Question: Why is patch embedding in ViT NOT just flattening?

- A. It reduces image size
- B. It normalizes data
- C. It is a learnable projection using convolution filters
- D. It removes channels
- E. It uses pooling

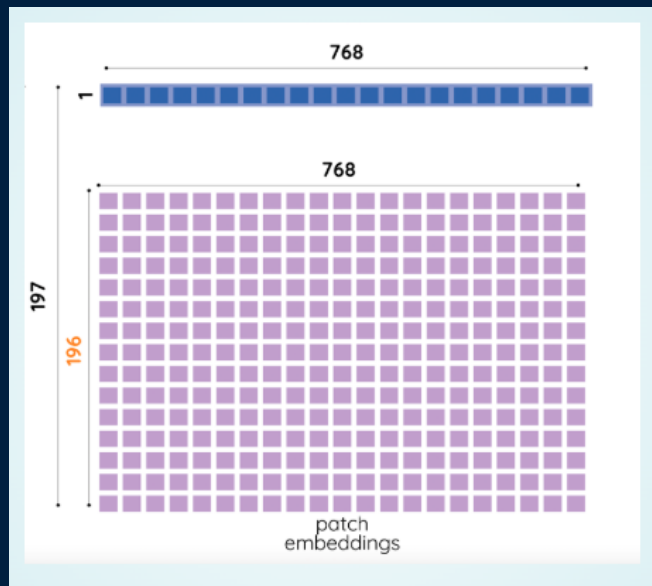
CLS Token

Why Do We Need [CLS]?

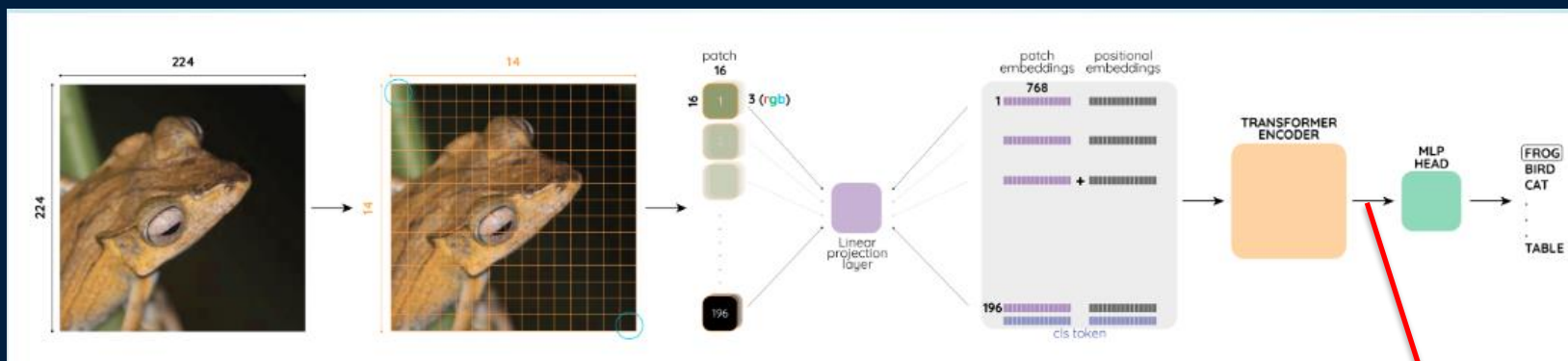
Transformers output one vector per token.

For classification, we need one vector for the whole image.

The [CLS] token collects information from all patches via self-attention.



Transformer Encoder



Classification Head

- Take the final [CLS] token
- Feed into an MLP
- Output class probabilities

ViT-Base

```
class ViT(pl.LightningModule):
    def __init__(self, L, num_classes=1000):
        super().__init__()

        # Patch embedding
        self.patch_embed = nn.Conv2d(3, 768, kernel_size=16, stride=16)

        # CLS + positional embeddings
        self.cls_token = nn.Parameter(torch.zeros(1, 1, 768))
        self.pos_embed = nn.Embedding(197, 768)

        # Transformer encoder
        encoder = nn.TransformerEncoderLayer(d_model=768, nhead=8)
        self.encoder = nn.TransformerEncoder(encoder, num_layers=L)

        # Classification head
        self.classifier = nn.Linear(768, num_classes)

    def forward(self, x):
        B = x.size(0)

        # Patch embeddings → (B, 196, 768)
        x = self.patch_embed(x).flatten(2).transpose(1, 2)

        # Add CLS token → (B, 197, 768)
        cls = self.cls_token.expand(B, -1, -1)
        x = torch.cat([cls, x], dim=1)

        # Add positional embeddings
        x = x + self.pos_embed(torch.arange(197, device=x.device))

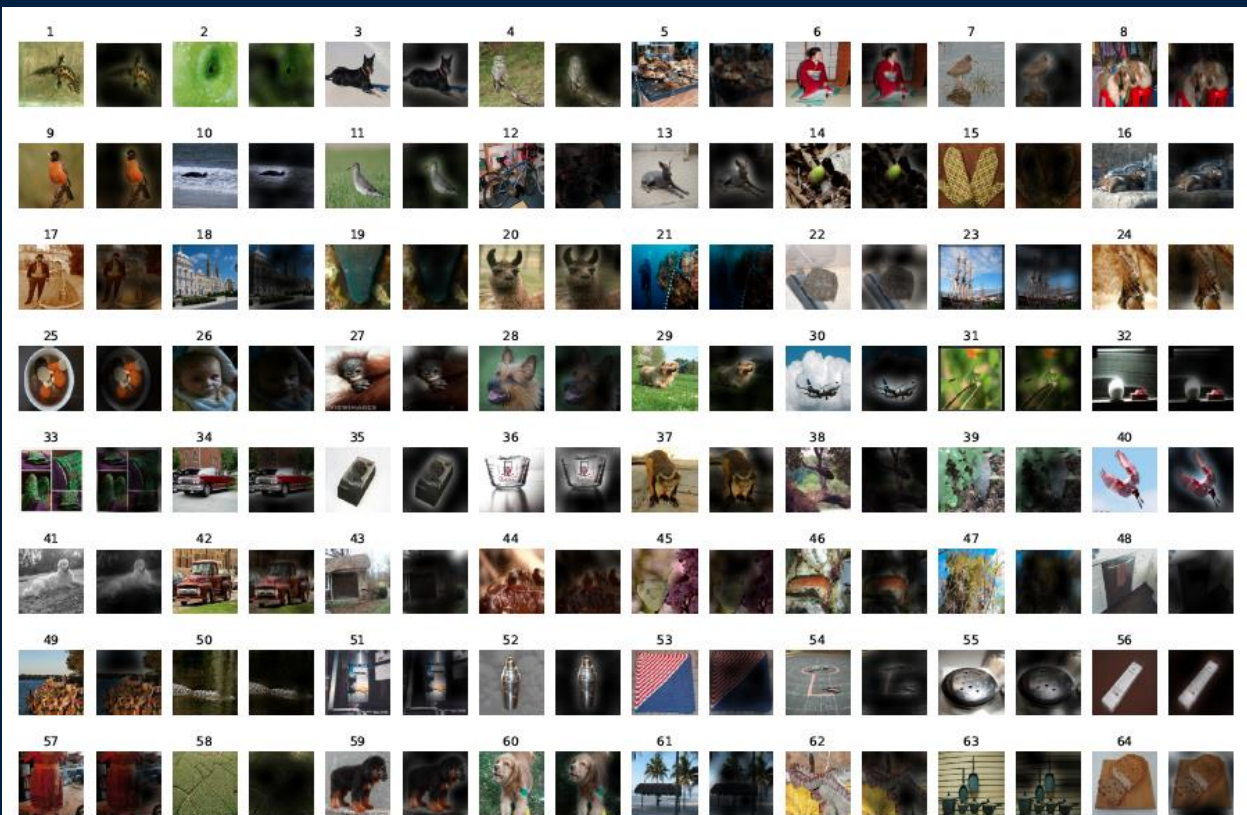
        # Transformer encoder
        x = self.encoder(x)

        # Classification from CLS token
        return self.mlp_head(x[:, 0])
```

ViT Model Variants

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Attention



From CNN Detectors to Transformers

Traditional object detectors (Faster R-CNN, YOLO):

- Use **anchors**
- Require **Non-Max Suppression (NMS)**
- Complex pipeline

Problems:

- Many hand-designed components
- Hard to optimize end-to-end

DEtection TRansformer : DETR

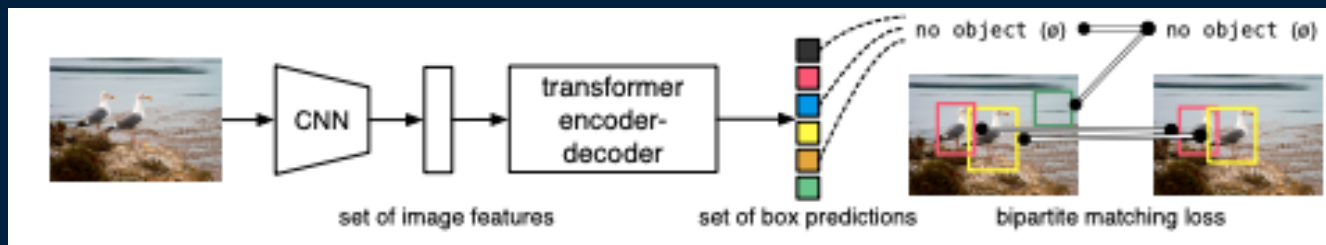
Predict a **fixed set of objects**

No anchors, no NMS

Each prediction = (class, bounding box)

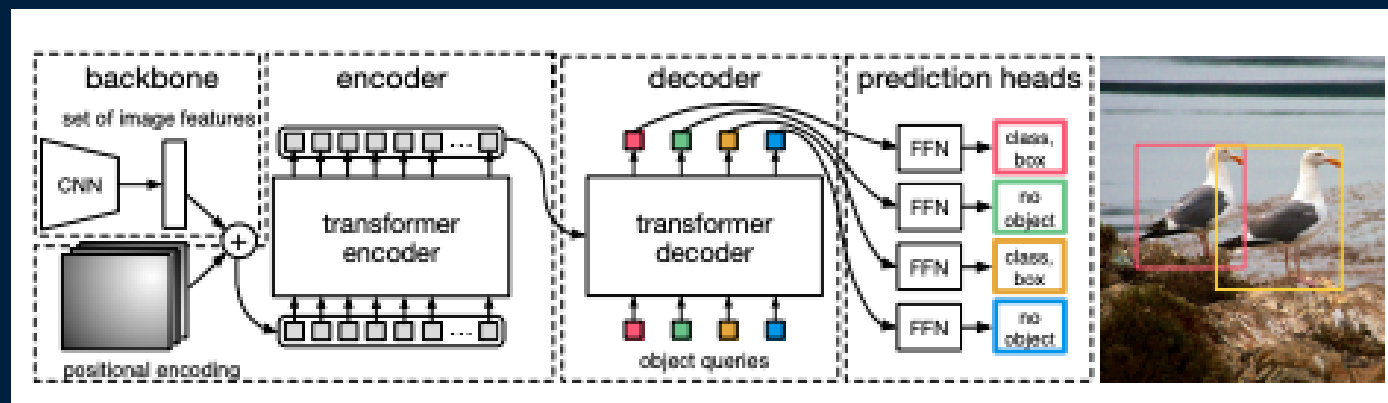
Output = $\{(c_1, b_1), (c_2, b_2), \dots, (c_N, b_N)\}$

Use **bipartite matching** to match predictions to ground truth

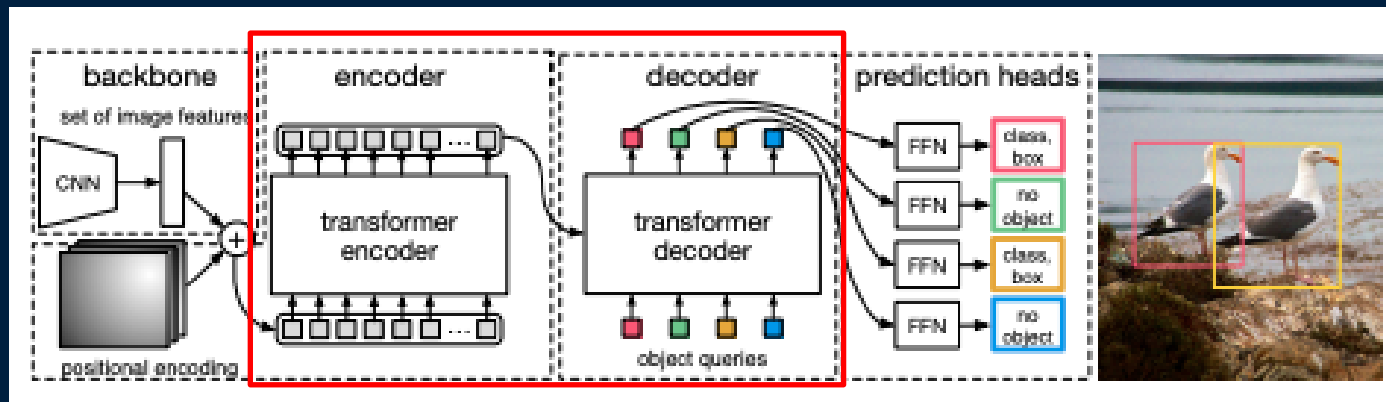


Carion + End-to-End Object Detection with Transformers: ECCV 2020

DETR Architecture Overview

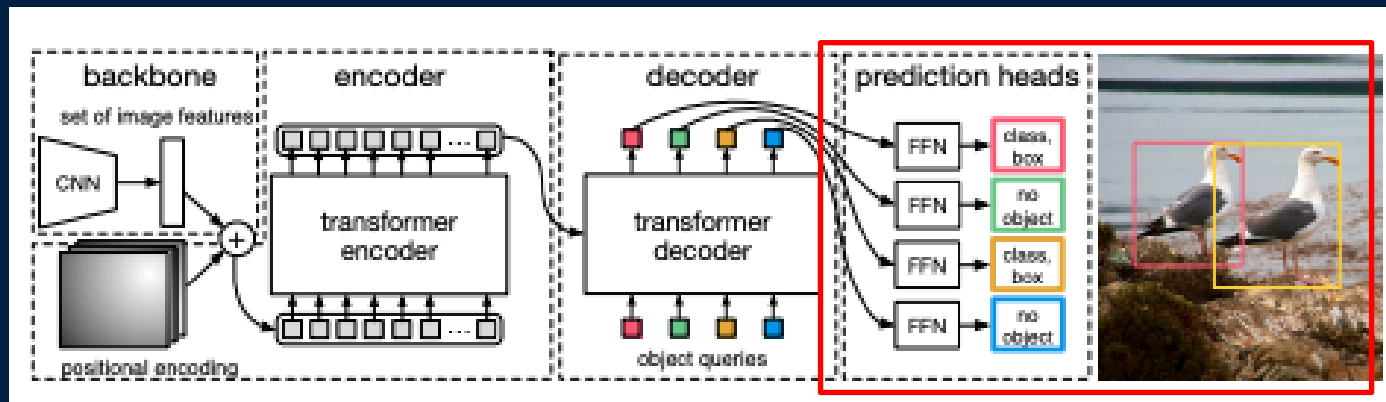


DETR Architecture Overview

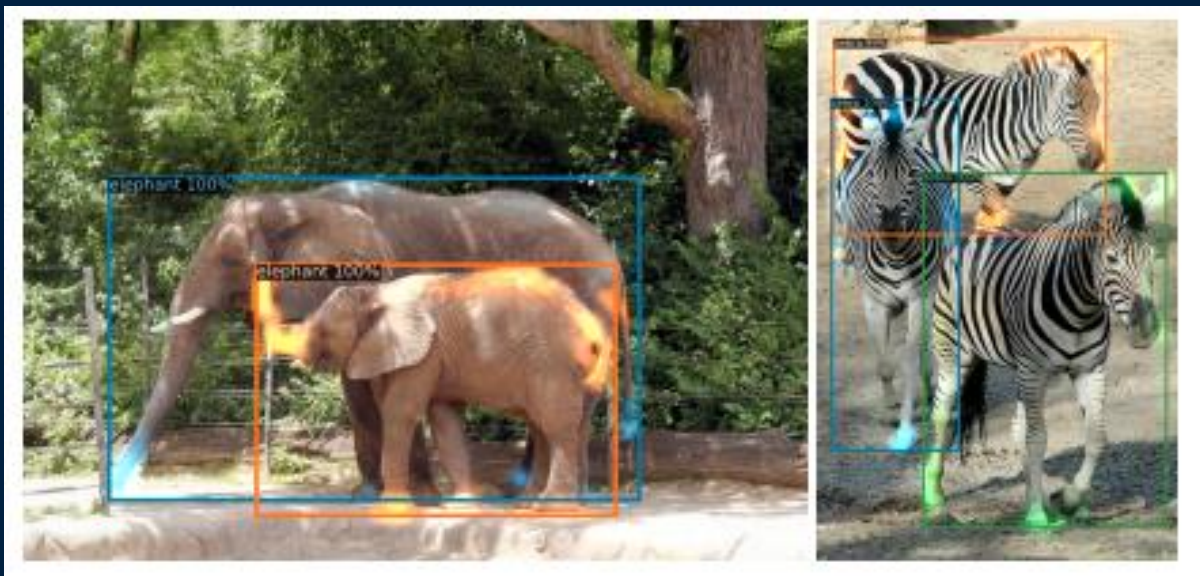


A transformer first encodes the given input, and then the decoder takes as input a small fixed number of learned positional embeddings, which we call object queries, and additionally attends to the encoder output.

DETR Architecture Overview



Each output embedding of the decoder is passed to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a "no object" class.



Question

Question: What is the key innovation of DETR compared to traditional detectors?

- A. Uses object queries and removes anchors and NMS
- B. Uses more filters
- C. Uses object queries, keys and values
- D. Uses smaller images
- E. Uses only MLP

Objectives

By the end of this lecture, students should be able to:

- Understand how **self-attention works for images** and how Q, K, V are computed
- Explain the difference between **CNN feature extraction** and **Transformer-based modeling**
- Describe the role of **multi-head attention** in capturing different relationships
- Understand the **Transformer encoder structure** (Attention + MLP + residuals)
- Explain how **Vision Transformers (ViT)** convert images into patch embeddings
- Describe how **[CLS] token** is used for classification
- Understand how **DETR reformulates object detection as a set prediction problem**
- Explain the role of **object queries and Transformer decoder in DETR**



THE UNIVERSITY OF BRITISH COLUMBIA

